TripStack Capstone

Participant Assignment & Question Paper

Week 7 · Days 5 & 6 · Final Capstone · TripStack (live hosted booking site)

Jai Singh

This is your **exam brief**. It tells you (1) what you must produce, (2) **your individual scenario** — mapped to *your* employee ID, and (3) **exactly where and how your employee ID is used** in your suite and in the Day-6 demo.

> Everyone does the **same engineering, individually**, against a **different booking scenario** and a **different set of injected conditions**. Your card is unique — it kills copy-paste and makes your demo worth watching.

1 · What you must produce (Day 5)

One repository, two projects, one CI/CD pipeline, run green against the live site:

Project	Tech	Covers
UI	**Playwright (TypeScript)**	the booking journeys, role-first locators, dynamic XPath where structural
API + DB	**RestAssured + JDBC (Java, JUnit 5)** — build with Maven or Gradle (your choice)	API contracts + database assertions + BOLA/auth negatives

Your suite must cover **UI + API + DB + resilience + performance + security**, be wired into **one GitHub Actions CI/CD pipeline** (two jobs, tagged parallel runs, **security & performance gates**, Allure + traces as artifacts, merge-gated), and handle **secret test data** (resolved at runtime, masked everywhere, never in the repo or the demo).

How you are graded (100 pts)

Criterion	Points
Framework structure & quality	**10**
Locator strategy	**10**
Integration completeness — UI + API + DB	**10**
Resilience under injected fault	**10**
Security — access-control & auth negatives	**10**
Performance — baseline + threshold gate	**10**
CI/CD pipeline	**10**
Secret test-data handling	**5**
Locator resilience to the live v2 DOM	**5**
Individual viva	**20**
Total	**100**

> **Performance and CI/CD each carry 10 marks and are exercised live on Day 6** — see §4: you get a **3-hour hands-on block** to finalize your CI/CD pipeline and your performance baseline/threshold against the live site before your demo slot.
>
> The **viva carries 20 marks** — the single largest block. Your live diagnosis and your answers to *why* (why this locator, why this assertion, where you would **not** automate, how you traced the injected fault, how you read the perf trace) weigh as much as the code. Prepare to explain, not just demonstrate.

2 · Your assignment card

Find your row **by your name**. **Your login is the email in your row; the password for every account is `Password@123`.** Your employee ID is the number in the ID column (it is embedded in your login token — see §3).

Date offset = book for **today + N days** (keeps everyone on a distinct date).

#	Participant	Emp ID	Login (email)	Track	Journey	Route (+days)	Day-6 fault	Perf target	Security negative	v2 DOM change
E01	**Ajay**	**1001**	alice@tripstack.test	T1	Flight · one-way · cabin	DEL → BLR +21	Payment latency	Flight search	BOLA (read other's PNR)	Class rename
E02	**Akhil**	**1002**	bob@tripstack.test	T1	Bus · seater · deck	BOM → DEL +30	Payment 500	Seat-map render	Cancel another's booking	Tag change (span→div)
E03	**Aravind**	**1003**	carol@tripstack.test	T1	Flight · multi-city · cabin	BLR → GOI +14	Gateway timeout	My Trips	Privilege escalation	New wrapper element
E04	**Athuldev**	**1004**	dave@tripstack.test	T1	Bus · sleeper · deck	HYD → CCU +10	Connection reset	Results listing	Tampered JWT	Attribute rename
E05	**Bhumika**	**1005**	erin@tripstack.test	T1	Flight · round-trip · cabin	PUN → BOM +7	Seat-hold expiry	Checkout	Expired token	Node re-order
E06	**Chaithra**	**1006**	frank@tripstack.test	T1	Bus · AC-semi · deck	DEL → BOM +18	Payment decline (402)	Flight search	BOLA (read other's PNR)	Class rename
E07	**Chandana**	**1007**	grace@tripstack.test	T1	Flight · one-way · cabin	MAA → HYD +25	Payment latency	Seat-map render	Cancel another's booking	Tag change (span→div)
E08	**Cipiraj**	**1008**	heidi@tripstack.test	T1	Bus · seater · deck	CCU → DEL +12	Payment 500	My Trips	Privilege escalation	New wrapper element
E09	**Deepak**	**1009**	ivan@tripstack.test	T1	Flight · multi-city · cabin	BLR → MAA +9	Gateway timeout	Results listing	Tampered JWT	Attribute rename
E10	**George**	**1010**	judy@tripstack.test	T1	Bus · sleeper · deck	AMD → DEL +16	Connection reset	Checkout	Expired token	Node re-order
E11	**Hemanth**	**1011**	karl@tripstack.test	T2	Flight · round-trip · cabin	JAI → BOM +28	Gateway timeout	Results listing	Privilege escalation	Attribute rename
E12	**Jamesy**	**1012**	laura@tripstack.test	T2	Bus · AC-semi · deck	GOI → PUN +11	Connection reset	Checkout	Tampered JWT	Node re-order
E13	**Justin**	**1013**	mallory@tripstack.test	T2	Flight · one-way · cabin	LKO → DEL +20	Seat-hold expiry	Flight search	Expired token	Class rename
E14	**Jyothsna**	**1014**	niaj@tripstack.test	T2	Bus · seater · deck	IXC → BLR +15	Payment decline (402)	Seat-map render	BOLA (read other's PNR)	Tag change (span→div)
E15	**Karthikeyan**	**1015**	olivia@tripstack.test	T2	Flight · multi-city · cabin	COK → BOM +22	Payment latency	My Trips	Cancel another's booking	New wrapper element
E16	**Kavya**	**1016**	peggy@tripstack.test	T2	Bus · sleeper · deck	DEL → IXC +13	Payment 500	Results listing	Privilege escalation	Attribute rename
E17	**Lahari**	**1017**	quinn@tripstack.test	T2	Flight · round-trip · cabin	BOM → GOI +8	Gateway timeout	Checkout	Tampered JWT	Node re-order
E18	**Lakhan**	**1018**	rupert@tripstack.test	T2	Bus · AC-semi · deck	BLR → HYD +17	Connection reset	Flight search	Expired token	Class rename
E19	**ManiPrasad**	**1019**	sybil@tripstack.test	T2	Flight · one-way · cabin	CCU → BOM +24	Seat-hold expiry	Seat-map render	BOLA (read other's PNR)	Tag change (span→div)

#	Participant	Emp ID	Login (email)	Track	Journey	Route (+days)	Day-6 fault	Perf target	Security negative	v2 DOM change
E20	**Manjushree**	**1020**	trent@tripstack.test	T2	Bus · seater · deck	HYD → DEL +19	Payment decline (402)	My Trips	Cancel another's booking	New wrapper element
E21	**Monisha**	**1021**	uma@tripstack.test	T3	Flight · multi-city · cabin	MAA → BLR +6	Seat-hold expiry	Seat-map render	Expired token	Tag change (span→div)
E22	**Muhammed**	**1022**	victor@tripstack.test	T3	Bus · sleeper · deck	DEL → JAI +26	Payment decline (402)	My Trips	BOLA (read other's PNR)	New wrapper element
E23	**Premchand**	**1023**	wendy@tripstack.test	T3	Flight · round-trip · cabin	PUN → DEL +23	Payment latency	Results listing	Cancel another's booking	Attribute rename
E24	**Ragumaan**	**1024**	xavier@tripstack.test	T3	Bus · AC-semi · deck	GOI → BLR +10	Payment 500	Checkout	Privilege escalation	Node re-order
E25	**Rishika**	**1025**	yvonne@tripstack.test	T3	Flight · one-way · cabin	AMD → BOM +15	Gateway timeout	Flight search	Tampered JWT	Class rename
E26	**Saiteja**	**1026**	zara@tripstack.test	T3	Bus · seater · deck	LKO → BLR +27	Connection reset	Seat-map render	Expired token	Tag change (span→div)
E27	**Salman**	**1027**	aaron@tripstack.test	T3	Flight · multi-city · cabin	BOM → MAA +9	Seat-hold expiry	My Trips	BOLA (read other's PNR)	New wrapper element
E28	**Sankaran**	**1028**	bianca@tripstack.test	T3	Bus · sleeper · deck	DEL → COK +29	Payment decline (402)	Results listing	Cancel another's booking	Attribute rename
E29	**Shahbaz**	**1029**	chetan@tripstack.test	T3	Flight · round-trip · cabin	BLR → CCU +12	Payment latency	Checkout	Privilege escalation	Node re-order
E30	**Vikram**	**1030**	divya@tripstack.test	T3	Bus · AC-semi · deck	HYD → BOM +14	Payment 500	Flight search	Tampered JWT	Class rename

Tracks run in parallel on Day 6 (T1 = E01–E10, T2 = E11–E20, T3 = E21–E30). By design, no two people demoing side by side share a fault, perf target, security negative, or v2 change — so a neighbour's injection can never break your demo, and you cannot copy the person before you.

> **Special accounts (shared tools, not your journey):** `alice` (1001) is the **director's** control-plane/admin account. `bob` (1002) and `carol` (1003) are **viewer**-role accounts — use a viewer login as the "low-privilege" actor for a **privilege-escalation** test. Every account can run the booking journey.

3 · Where & how you use your Employee ID

This is the spine of the capstone. Read it carefully.

1. You never type your employee ID into the UI. You **log in** with your assigned email + `Password@123`. Your employee ID is carried **inside your login token (JWT)** and resolved by the app on every request. Logging in *is* how you "use" your employee ID.

2. Your data is namespaced by your employee ID.

- Every booking you make is stamped with **your** employee ID.
- Your PNR is formatted **`TS-<yourEmpId>-<seq>`** — e.g. employee 1004's first PNR is `TS-1004-0001`. Assert this format in UI *and* API.
- "My bookings" returns **only your** namespace. In your **JDBC/DB test**, assert the booking row's namespace/`emp_id` column equals **your employee ID** — that column is the ownership boundary.

3. The director injects your card against your employee ID. During your demo slot, the director flips feature flags **keyed to your employee ID** through the director-only control plane. They affect **only your traffic** — your neighbours are untouched. What the director sets for you:

Your card says…	Director sets, keyed to your empId	You observe / assert
Day-6 fault	a per-employee fault flag (latency / 500 / timeout / connection-reset / decline / seat-hold-expiry)	your resilience test detects it; you diagnose the root cause; the circuit breaker opens then recovers
Perf target	a per-employee slow-path flag on your named endpoint	your performance gate fails on the regression; you explain it against the OpenTelemetry trace
v2 DOM change	a per-employee DOM = v2 flag	your session is served the changed markup; robust locators still pass, brittle ones you fix live

You do **not** operate the control plane — that is the director's job. **You observe the effect** on your employee's traffic and prove your suite reacts.

4. Your security negatives use your employee ID vs. another one.

- **BOLA / cross-namespace read:** logged in as your employee, request **another employee's** PNR → expect **403**. (The other emp is a different employee ID.)
- **Cancel another's booking:** attempt to cancel a booking that belongs to a **different** employee ID → expect **403**.
- **Privilege escalation:** log in as a **viewer** account and hit a privileged (admin) route → expect **403**.
- **Expired / tampered token:** call the API with an expired or altered token → expect **401**. (Tokens are short-lived by design.)

5. Keep your namespace clean. Use the **self-serve reset** (keyed to *your* employee ID) to clear your bookings between runs, so repeated tests stay deterministic and you never collide with your earlier data.

> **In one line:** *log in as your employee → book under your namespace (PNR **`TS-<empId>-…`**) → the director flips your card's flags against your empId → your tests assert on your empId (namespace/PNR) and against other empIds (BOLA/priv-esc).*

4 · Day 6 — CI/CD + Performance, then Demo + Viva

Day 6 runs in two parts.

Part A · 3-hour hands-on block (CI/CD + Performance)

A **3-hour** working window against the **live site** to finalize the two things that are proven live today. Everyone works in parallel; the director enables per-employee flags on request during your demo slot in Part B.

1. **CI/CD (10 marks)** — get your **one GitHub Actions pipeline** green: two jobs (Playwright UI + RestAssured/JDBC API) running **tagged tests in parallel**, a **security gate** and a **performance gate**, Allure + Playwright traces published as artifacts, and the **merge blocked unless green**. Push a real commit and show the run.
2. **Performance (10 marks)** — establish a **baseline** for *your* named endpoint (see your card's **Perf target**), set a **threshold gate** that fails the build on a regression, and be ready to **read the OpenTelemetry trace** to say *where* the latency came from. In Part B the director injects **your slow-path** so the gate fires live.

> Both of these are **wired into the same pipeline** as the rest of your suite — CI/CD is the harness; performance is one of the gates it enforces.

Part B · Demo + Viva (≈25 min per person: ~10 demo + ~15 viva)

The **viva is the largest single block** (20 of 100 marks). Show these seven beats live, then expect to be questioned on every one of them.

1. **Live booking, end-to-end** — your journey → PNR asserted in UI (Playwright), API (RestAssured) and DB (JDBC). Role-first locators; dynamic XPath only where structural.
2. **State-machine negatives** — the invalid transitions from your card return the right codes (hold-expired, cancel-twice, non-refundable, decline→breaker, seat-conflict, BOLA).
3. **Resilience** — the director injects **your fault**; your suite detects it, you **diagnose the root cause**, the breaker opens and recovers.
4. **Performance** — the director injects **your slow-path**; your gate fails on the regression and you **explain it against the OpenTelemetry trace**.
5. **Security** — **your** security negative passes (e.g. 403 on another employee's PNR); auth negatives pass.
6. **Locator resilience** — the **v2 DOM** is deployed for your employee; robust locators keep passing, any brittle one you **diagnose and fix live**.
7. **CI/CD + judgement (viva)** — your green pipeline run and Allure artifact, then the individual viva: *where would you NOT automate? why this locator? how did you trace the injected fault to its root cause? how did you read the perf trace? what would you assert differently?*

> **Secret discipline is graded here too:** nothing secret may appear on screen or in your recording — not in a field, the console, the network tab, an Allure attachment or a screenshot. Record a dry-run and watch it back.

5 · Reference

- **Live site — use whichever your network reaches (same site, same data on both):**
  - **Primary (whitelisted):** `https://tripstack.doomple.com` — web UI · `api.` · `admin.` subdomains.
  - **Fallback:** `https://tripstack-747773947804.asia-south1.run.app` — single host, surfaces by **path**: `/` UI · `/api` reference · `/openapi.yaml` spec · `/admin`. Use this only if the primary domain is unreachable on your network.
  - Both front-ends share one database and one identity, so your login, bookings, PNRs, and the director's injected flags are identical on either URL.
- **Login:** your assigned email + `Password@123`. Your employee ID is in your token — never typed into a form.
- **OpenAPI spec:** `<base>/openapi.yaml` (your RestAssured / contract source of truth).
- **Reset your data:** self-serve reset endpoint keyed to your employee ID.
- **You get on the day:** this card, your login, the two URLs above, and the rules of engagement.

- **Rule of engagement:** your tools touch **only TripStack**, and **only your namespace** (except the deliberate cross-namespace 403 negative). Anything else is a fail. No AI in the loop — every diagnosis and explanation is your own.